

Event Delegation – Answers to Tasks 1–12

Plain HTML + JavaScript solutions using a single parent click listener. The key idea is to listen on the parent container and use `event.target` (or `closest()`) to identify the clicked remove button.

Core pattern: `parent.addEventListener('click', (event) => { if (event.target.matches('.remove-btn')) { ... } });`

Task 1: Song cards – remove only the clicked card

```
<!DOCTYPE html>
<html>
<head>
  <title>Song Cards</title>
  <style>
    .song-card { padding: 12px; margin: 8px 0; border: 1px solid #ccc; }
    .remove-btn { float: right; cursor: pointer; }
  </style>
</head>
<body>
  <div id="songList">
    <div class="song-card">
      <span>Blinding Lights</span>
      <button class="remove-btn">X</button>
    </div>
    <div class="song-card">
      <span>Perfect</span>
      <button class="remove-btn">X</button>
    </div>
    <div class="song-card">
      <span>Shape of You</span>
      <button class="remove-btn">X</button>
    </div>
  </div>

  <script>
    const songList = document.getElementById("songList");

    songList.addEventListener("click", function (event) {
      if (event.target.classList.contains("remove-btn")) {
        event.target.closest(".song-card").remove();
      }
    });
  </script>
</body>
</html>
```

Explanation: Only one listener is attached to `#songList`. The clicked X is found through `event.target`, and `closest('.song-card')` identifies the correct card.

Task 2: Song cards – same requirement using `closest()`

```
<!DOCTYPE html>
<html>
<body>
  <div id="songs">
    <div class="song-card">
      <span>Love Story</span>
      <button class="remove-btn">X</button>
    </div>
    <div class="song-card">
      <span>Faded</span>
      <button class="remove-btn">X</button>
    </div>
    <div class="song-card">
      <span>Levitating</span>
      <button class="remove-btn">X</button>
    </div>
  </div>

  <script>
    const songs = document.querySelector("#songs");

    songs.addEventListener("click", (event) => {
      const button = event.target.closest(".remove-btn");
    });
  </script>
</body>
</html>
```

```

    if (!button) return;

    const card = button.closest(".song-card");
    card.remove();
  });
</script>
</body>
</html>

```

Explanation: This version is especially useful when the button contains an icon or span. `closest('.remove-btn')` still finds the button even when the inner element is the original event target.

Task 3: Favorite songs list – Remove button with one listener

```

<!DOCTYPE html>
<html>
<body>
  <h2>Favorite Songs</h2>

  <ul id="favoriteSongs">
    <li>Believer <button class="remove-btn">Remove</button></li>
    <li>Havana <button class="remove-btn">Remove</button></li>
    <li>Senorita <button class="remove-btn">Remove</button></li>
  </ul>

  <script>
    const favoriteSongs = document.getElementById("favoriteSongs");

    favoriteSongs.addEventListener("click", (event) => {
      if (event.target.classList.contains("remove-btn")) {
        event.target.closest("li").remove();
      }
    });
  </script>
</body>
</html>

```

Explanation: The parent `ul` receives every click through event bubbling. No listener is needed on each individual button.

Task 4: Flipkart-style wishlist – add products dynamically and remove them

```

<!DOCTYPE html>
<html>
<head>
  <style>
    .product-card {
      border: 1px solid #ddd; padding: 15px; margin: 10px;
      width: 250px; border-radius: 6px;
    }
    .remove-btn { cursor: pointer; }
  </style>
</head>
<body>
  <button id="addProduct">Add Product</button>
  <div id="wishlist">
    <div class="product-card">
      <h3>Wireless Headphones</h3>
      <p>■1,999</p>
      <button class="remove-btn">Remove</button>
    </div>
    <div class="product-card">
      <h3>Smart Watch</h3>
      <p>■2,499</p>
      <button class="remove-btn">Remove</button>
    </div>
  </div>

  <script>
    const wishlist = document.getElementById("wishlist");
    const addProduct = document.getElementById("addProduct");

    wishlist.addEventListener("click", (event) => {
      if (event.target.classList.contains("remove-btn")) {
        event.target.closest(".product-card").remove();
      }
    });
  </script>

```

```

    }
  });

  addProduct.addEventListener("click", () => {
    const card = document.createElement("div");
    card.className = "product-card";

    card.innerHTML = `
      <h3>New Product</h3>
      <p>■999</p>
      <button class="remove-btn">Remove</button>
    `;

    wishlist.appendChild(card);
  });
</script>
</body>
</html>

```

Explanation: The dynamically created card needs no new remove listener because the click bubbles to the existing wishlist listener.

Task 5: Flipkart-style wishlist – another dynamic implementation

```

<!DOCTYPE html>
<html>
<body>
  <button id="addProduct">Add Product</button>

  <div id="wishlist">
    <div class="product-card">
      <h3>Mobile Phone</h3>
      <button class="remove-btn">Remove</button>
    </div>
  </div>

  <script>
    const wishlist = document.querySelector("#wishlist");
    const addProduct = document.querySelector("#addProduct");

    // One listener handles old and future cards.
    wishlist.addEventListener("click", (event) => {
      const removeButton = event.target.closest(".remove-btn");

      if (!removeButton) return;

      removeButton.closest(".product-card").remove();
    });

    addProduct.addEventListener("click", () => {
      const product = document.createElement("div");
      product.className = "product-card";

      product.innerHTML = `
        <h3>Random Product</h3>
        <button class="remove-btn">Remove</button>
      `;

      wishlist.appendChild(product);
    });
  </script>
</body>
</html>

```

Explanation: closest() makes the handler robust when the remove button contains nested elements such as an SVG icon or span.

Task 6: Flipkart-style shopping cart – remove using event bubbling

```

<!DOCTYPE html>
<html>
<body>
  <div id="cart">
    <div class="cart-card">
      <h3>Laptop</h3>
      <p>■55,000</p>

```

```

    <button class="remove-btn">X</button>
  </div>

  <div class="card-card">
    <h3>Keyboard</h3>
    <p>■2,000</p>
    <button class="remove-btn">X</button>
  </div>

  <div class="card-card">
    <h3>Mouse</h3>
    <p>■800</p>
    <button class="remove-btn">X</button>
  </div>
</div>

<script>
  const card = document.getElementById("card");

  card.addEventListener("click", (event) => {
    if (!event.target.classList.contains("remove-btn")) return;

    const card = event.target.closest(".card-card");
    card.remove();
  });
</script>
</body>
</html>

```

Explanation: Clicking the X first targets the button, then the event bubbles to #card. The parent listener removes only that button's card.

Task 7: Instagram story cards – refactor individual listeners to one parent listener

```

// BEFORE: one listener for every card
document.querySelectorAll(".story-card").forEach((card) => {
  card.querySelector(".remove-btn").addEventListener("click", () => {
    card.remove();
  });
});

// AFTER: event delegation
const storyContainer = document.querySelector("#stories");

storyContainer.addEventListener("click", (event) => {
  const removeButton = event.target.closest(".remove-btn");

  if (!removeButton) return;

  removeButton.closest(".story-card").remove();
});

```

Explanation: The refactored version satisfies the constraint: only one click listener is used for all story cards.

Task 8: Instagram story cards – complete delegated version

```

<!DOCTYPE html>
<html>
<body>
  <div id="stories">
    <div class="story-card">
      
      <button class="remove-btn">X</button>
    </div>

    <div class="story-card">
      
      <button class="remove-btn">X</button>
    </div>

    <div class="story-card">
      
      <button class="remove-btn">X</button>
    </div>
  </div>

```

```

</div>

<script>
const stories = document.getElementById("stories");

stories.addEventListener("click", (event) => {
  if (!event.target.closest(".remove-btn")) return;

  const storyCard = event.target.closest(".story-card");

  if (storyCard) {
    storyCard.remove();
  }
});
</script>
</body>
</html>

```

Explanation: All cards share the same parent listener. Adding another story card later will automatically make its remove button work.

Task 9: Cart – dynamically added product still has working X

```

<!DOCTYPE html>
<html>
<body>
  <button id="addRandom">Add Random Product</button>

  <div id="cart">
    <div class="cart-card">
      <h3>Existing Product</h3>
      <button class="remove-btn">X</button>
    </div>
  </div>

  <script>
    const cart = document.getElementById("cart");
    const addRandom = document.getElementById("addRandom");

    // This is the only listener for remove buttons.
    cart.addEventListener("click", (event) => {
      const button = event.target.closest(".remove-btn");

      if (!button) return;

      button.closest(".cart-card").remove();
    });

    addRandom.addEventListener("click", () => {
      const card = document.createElement("div");
      card.className = "cart-card";

      card.innerHTML = `
        <h3>Random Product</h3>
        <button class="remove-btn">X</button>
      `;

      cart.appendChild(card);
    });
  </script>
</body>
</html>

```

Explanation: Event delegation is ideal here because the listener is attached to a stable parent, while product cards can be created and removed at any time.

Task 10: YouTube-style notification feed – fade out and remove

```

<!DOCTYPE html>
<html>
<head>
  <style>
    .notification {
      padding: 12px;
      margin: 8px 0;
      border: 1px solid #ddd;
      transition: opacity 0.4s ease;
    }
  </style>

```

```

    }

    .notification.fade-out {
      opacity: 0;
    }
  </style>
</head>
<body>
  <div id="notifications">
    <div class="notification">
      New video from Tech Channel
      <button class="dismiss-btn">Dismiss</button>
    </div>

    <div class="notification">
      Someone liked your comment
      <button class="dismiss-btn">Dismiss</button>
    </div>
  </div>

  <button id="loadNotification">Load Notification</button>

  <script>
    const notifications = document.getElementById("notifications");
    const loadNotification = document.getElementById("loadNotification");

    notifications.addEventListener("click", (event) => {
      const button = event.target.closest(".dismiss-btn");

      if (!button) return;

      const card = button.closest(".notification");
      card.classList.add("fade-out");

      card.addEventListener("transitionend", () => {
        card.remove();
      }, { once: true });
    });

    // Simulate dynamically loaded notifications.
    loadNotification.addEventListener("click", () => {
      const card = document.createElement("div");
      card.className = "notification";

      card.innerHTML = `
        New dynamically loaded notification
        <button class="dismiss-btn">Dismiss</button>
      `;

      notifications.appendChild(card);
    });
  </script>
</body>
</html>

```

Explanation: The notification parent handles both initial and dynamically loaded cards. The card first receives a fade-out class, then is removed after the CSS transition ends.

Task 11: YouTube-style notification feed – delegated dismiss action

```

const notificationFeed = document.querySelector("#notifications");

notificationFeed.addEventListener("click", (event) => {
  const dismissButton = event.target.closest(".dismiss-btn");

  if (!dismissButton) return;

  const notification = dismissButton.closest(".notification");

  notification.classList.add("fade-out");

  setTimeout(() => {
    notification.remove();
  }, 400);
});

```

Explanation: This compact version uses a 400 ms timeout matching the CSS transition duration. The same delegated listener works for notifications inserted later.

Task 12: Fix the remove-button bug with event delegation

```
// BUGGY CODE:
// document.querySelectorAll('.remove-btn').forEach(btn => {
//   btn.addEventListener('click', function() {
//     this.parentElement.remove();
//   });
// });

// FIXED CODE:
const container = document.querySelector("#cards");

container.addEventListener("click", (event) => {
  const removeButton = event.target.closest(".remove-btn");

  if (!removeButton) return;

  const card = removeButton.closest(".card");

  if (card) {
    card.remove();
  }
});
```

Explanation: Why the fix works: `querySelectorAll` only attaches listeners to buttons that exist at the time the code runs. The parent listener remains active and receives clicks from dynamically added buttons through event bubbling.

Quick Revision – Event Delegation

1. Attach one listener to a stable parent.
2. Let the click bubble from the button to the parent.
3. Use `event.target` or `event.target.closest(...)` to identify the clicked control.
4. Use `closest(...)` again to find the card/list item that should be removed.
5. Because the parent listener already exists, dynamically inserted elements work automatically.

Most reusable pattern:

```
const parent = document.querySelector('#parent');

parent.addEventListener('click', (event) => {
  const button = event.target.closest('.remove-btn');
  if (!button) return;
  button.closest('.card').remove();
});
```